

Chapter 3

Predicting Outcomes Using Bayes

IN THIS CHAPTER

» Working with Bayes' Theorem in a basic way**» Performing tasks using networked Bayes****» Using Bayes for linear regression****» Using Bayes for logistic regression**

.....

Sometimes, a method of doing something seems less complicated than it actually is, which is how things went with Bayes' Theorem. The theorem receives its name from the Reverend Thomas Bayes, who discovered it in the 1740s and promptly discarded it. Later, Pierre Simon Laplace worked on it to give it its modern form, and then he discarded it as well.

According to Sharon McGrayne's book, *The Theory That Would Not Die*, many people used Bayes' Theorem for all sorts of purposes, from mounting a defense for Captain Dreyfus (see https://www.business-standard.com/article/markets/the-bayesian-curse-112011300083_1.html for details) to breaking the German Enigma code (see <https://www.investsmart.com.au/investment-news/the-theory-that-cracked-the-enigma-code/138342> for details), but they generally used it in secret. Not until the twenty-first century would anyone actually admit to using it for all the practical things it can do. So, this chapter is your introduction to a widely kept secret that can perform amazing practical tasks in statistics, and when you see it in its original form, you'll declare Bayes' Theorem to be surprisingly simple.

Of course, things never stay simple. The second part of this chapter moves into detailed uses of Bayes' Theorem by employing the networked version. The networked version uses the same basic form of the original version, but now you consider the complexity of evidence and the places that evidence can take you. Unsurprisingly, tracking down and employing all the evidence in an analysis can take time and sometimes become error prone. Court trials follow the same path. Yet, using Bayes helps you clear the muddy waters and make sense of difficult issues.

Book 3, [Chapters 1](#) and [2](#) help you discover both linear and logistic regression. The third part of this chapter helps you combine this knowledge with your newly acquired knowledge of Bayes' Theorem to create a pow-

erful method of performing data analysis by uniting two simpler methods.



REMEMBER You don't have to type the source code for this chapter manually. In fact, using the downloadable source is a lot easier. The source code for this chapter appears in the `DSPD_0303_Bayes.ipynb` source code file for Python and the `DSPD_R_0303_Bayes.ipynb` source code file for R. See the Introduction for details on how to find these source files.

Understanding Bayes' Theorem

Before you begin using Bayes' Theorem to perform practical tasks, knowing a little about its history is helpful. The reason this knowledge is so useful is because the theorem doesn't seem to be able to do everything it purports to do when you first see it, which is why many statisticians rejected it outright. After you do have a basic knowledge of how the theorem came into being, you need to look at the theorem itself. The following sections provide you with a history of Bayes' Theorem that then moves into the theorem itself. These sections discuss the theorem from a practical perspective.

Delving into Bayes history

You might wonder why anyone would name an algorithm Naïve Bayes (yet you find this algorithm among the most effective machine learning algorithms in packages such as Scikit-learn). The *naïve* part comes from its formulation; it makes some extreme simplifications to standard probability calculations. The reference to Bayes in its name relates to the Reverend Bayes and his theorem on probability.

The Reverend Thomas Bayes (1701–1761) was an English statistician and a philosopher who formulated his theorem during the first half of the eighteenth century. The theorem is based on a thought experiment and then a demonstration using the simplest of means. Reverend Bayes wanted to determine the probability of a future event based on the number of times it occurred in the past. It's hard to contemplate how to accomplish this task with any accuracy.

The demonstration relied on the use of two balls. An assistant would drop the first ball on a table where the end position of this ball was equally possible in any location, but not tell Bayes its location. The assistant would then drop a second ball, tell Bayes the position of the second ball, and then provide the position of the first ball relative to the location of this second ball. The assistant would then drop the second ball a number of additional times — each time telling Bayes the location of the second ball and the position of the first ball relative to the second. After each toss of the second ball, Bayes would attempt to guess the position of the first.

Eventually, he was to guess the position of the first ball based on the evidence provided by the second ball.

The theorem was never published while Bayes was alive. His friend Richard Price found Bayes' notes after his death in 1761 and published the material for Bayes, but no one seemed to read it at first. The theorem has deeply revolutionized the theory of probability by introducing the idea of conditional probability — that is, probability conditioned by evidence. The critics saw problems with Bayes' Theorem that you can summarize as follows:

- Guessing has no place in rigorous mathematics.
- If Bayes didn't know what to guess, he would simply assign all possible outcomes an equal probability of occurring.
- Using the prior calculations to make a new guess presented an insurmountable problem.

Often, it takes a problem to illuminate the need for a previously defined solution, which is what happened with Bayes' Theorem. By the late eighteenth century, the need to study astronomy and make sense of the observations made by the

- Chinese in 1100 BC
- Greeks in 200 BC
- Romans in AD 100
- Arabs in AD 1000

became essential. The readings made by these other civilizations not only reflected social and other biases but also were unreliable because of the differing methods of observation and the technology use. You might wonder why the study of astronomy suddenly became essential, and the short answer is money. Navigation of the late eighteenth century relied heavily on accurate celestial observations

(<https://penobscotmarinemuseum.org/pbho-1/history-of-navigation/navigation-18th-century>), so anyone who could make the readings more accurate could reduce the time required to ship goods from one part of the world to another.

Pierre-Simon Laplace wanted to solve the problem, but he couldn't just dive into the astronomy data without first having a means to dig through all that data to find out which was correct and which wasn't. He encountered Richard Price, who told him about Bayes' Theorem. Laplace used the theorem to solve an easier problem, that of the births of males and females. Some people had noticed that more boys than girls were born each year, but no proof existed for this observation. Laplace used Bayes' Theorem to prove that more boys are born each year than girls based on birth records. Other statisticians took notice and started using the theorem, often secretly, for a host of other calculations, such as the calculation of the masses of Jupiter and Saturn from a wide variety of observations by Alexis Bouvard (see <https://www.revolvy.com/page/Alexis-Bouvard> for details).

Considering the basic theorem

When thinking about Bayes' Theorem, it helps to start from the beginning — that is, probability itself. *Probability* tells you the likelihood of an event and is expressed in a numeric form. The probability of an event is measured in the range from 0 to 1 (from 0 percent to 100 percent) and it's empirically derived from counting the number of times a specific event happens with respect to all the events. You can calculate it from data!

When you observe events (for example, when a feature has a certain characteristic) and you want to estimate the probability associated with the event, you count the number of times the characteristic appears in the data and divide that figure by the total number of observations available. The result is a number ranging from 0 to 1, which expresses the probability.

When you estimate the probability of an event, you tend to believe that you can apply the probability in each situation. The term for this belief is *a priori* because it constitutes the first estimate of probability with regard to an event (the one that comes to mind first). For example, if you estimate the probability of an unknown person's being a female, you might say, after some counting, that it's 50 percent, which is the prior, or the first, probability that you will stick with.

The prior probability can change in the face of evidence, that is, something that can radically modify your expectations. For example, the evidence of whether a person is male or female could be that the person's hair is long or short. You can estimate having long hair as an event with 35 percent probability for the general population, but within the female population, it's 60 percent. If the percentage is higher in the female population, contrary to the general probability (the prior for having long hair), that's useful information for making a prediction.

Imagine that you have to guess whether a person is male or female and the evidence is that the person has long hair. This sounds like a predictive problem, and in the end, this situation is similar to predicting a categorical variable from data: We have a target variable with different categories and you have to guess the probability of each category based on evidence, the data. Reverend Bayes provided a useful formula:

$$P(B|E) = P(E|B) * P(B) / P(E)$$

The formula looks like statistical jargon and is a bit counterintuitive, so it needs to be explained in depth. Reading the formula using the previous example as input makes the meaning behind the formula quite a bit clearer:

- **P(B|E):** The probability of being a female (the belief B) given long hair (the evidence E). This part of the formula defines what you want to predict. In short, it says to predict y given x where y is an outcome (male or female) and x is the evidence (long or short hair).

- **P(E | B):** The probability of having long hair, the evidence of when a person is female. In this case, you already know that it's 60 percent. In every data problem, you can obtain this figure easily by simple cross-tabulation of the features against the target outcome.
- **P(B):** The probability of being a female, which has a 50 percent general chance (a prior).
- **P(E):** The probability of having long hair in general, which is 35 percent (another prior).

**TIP**

When reading parts of the formula such as $P(B | E)$, you should read them as follows: probability of B given E. The $|$ symbol translates as *given*. A probability expressed in this way is a conditional probability, because it's the probability of a belief, B, conditioned by the evidence presented by E. In this example, plugging the numbers into the formula translates into

$$60\% * 50\% / 35\% = 85.7\%$$

Therefore, getting back to the previous example, even if being a female is a 50 percent probability, just knowing evidence like long hair takes it up to 85.7 percent, which is a more favorable chance for the guess. You can be more confident in guessing that the person with long hair is a female because you have a bit less than a 15 percent chance of being wrong.

Using Naïve Bayes for Predictions

Even though Bayes' Theorem looks relatively simple compared to some of the other calculations you have already performed in this book, it really can do amazing things. The following sections discuss the basic theorem before you do anything fancy with it.

Finding out that Naïve Bayes isn't so naïve

Naïve Bayes, leveraging the simple Bayes' Theorem, takes advantage of all the evidence available in order to modify the prior base probability of your predictions. Because your data contains so much evidence — that is, it has many features — the data makes a big sum of all the probabilities derived from a simplified Naïve Bayes formula.

**REMEMBER**

As discussed in the “[Using Linear Regression to Guess Numbers](#)” section of Book 3, [Chapter 1](#), summing variables implies that the model takes them as separate and unique pieces of information. But this implication doesn't hold true in reality, because applications exist in a world of interconnections, with every piece of information connecting to many other pieces. Using one piece of information more than once means giving more emphasis to that particular piece.

Because you don't know (or you simply ignore) the relationships among each piece of evidence, you probably just plug all of them in to Naïve Bayes. The simple and naïve move of throwing everything that you know at the formula works well indeed, and many studies report good performance despite the fact that you make a naïve assumption. Using everything for prediction is okay, even though it seemingly shouldn't be given the strong association between variables. Here are some of the ways in which you commonly see Naïve Bayes used:

- Building spam detectors (catching all annoying emails in your inbox)
- Sentiment analysis (guessing whether a text contains positive or negative attitudes with respect to a topic, and detecting the mood of the speaker)
- Text-processing tasks such as spell correction, or guessing the language used to write or classify the text into a larger category



TIP

Naïve Bayes is also popular because it doesn't need as much data to work. It can naturally handle multiple classes. With some slight variable modifications (transforming them into classes), it can also handle numeric variables. Scikit-learn provides three Naïve Bayes classes in the `sklearn.naive_bayes` module:

- `MultinomialNB`: Uses the probabilities derived from a feature's presence. When a feature is present, it assigns a certain probability to the outcome, which the textual data indicates for the prediction.
- `BernoulliNB`: Provides the multinomial functionality of Naïve Bayes, but it penalizes the absence of a feature. This class assigns a different probability when the feature is present than when it's absent. In fact, it treats all features as dichotomous variables. (The distribution of a dichotomous variable is a Bernoulli distribution.) You can also use it with textual data.
- `GaussianNB`: Defines a version of Naïve Bayes that expects a normal distribution of all the features. Hence, this class is suboptimal for textual data in which words are sparse (use the multinomial or Bernoulli distributions instead). If your variables have positive and negative values, this is the best choice.

Predicting text classifications

Naïve Bayes is particularly popular for document classification. In textual problems, you often have millions of features involved, one for each word spelled correctly or incorrectly. Sometimes the text is associated with other nearby words in *n-grams*, that is, sequences of consecutive words. Naïve Bayes can learn the textual features quickly and provide fast predictions based on the input.

UNDERSTANDING ANACONDA NOTIFICATIONS

Newer versions of Anaconda display various notifications to help you understand why a particular process is taking longer than expected. For example, when you download the `20newsgroups` dataset, you may see the following message:

Downloading 20news dataset. This may take a few minutes.

Downloading dataset from

<https://ndownloader.figshare.com/files/5975967> (14 MB)

The message will often appear in the same background color and the same text characteristics as an error message. This message isn't an error; it simply gives you additional information about the progress of a task. If you don't see the message, it may simply mean that you have already downloaded the dataset at some point or that you're using an older version of Anaconda that doesn't support the message.

You could see other notifications for other needs. For example, when a package installation requires additional time to complete, you may see a notification message even if you have a copy of the package on your local system. All these messages help you understand what Anaconda is doing in the background.

This section tests text classifications using the binomial and multinomial Naïve Bayes models offered by Scikit-learn. The examples rely on the `20newsgroups` dataset, which contains a large number of posts from 20 kinds of newsgroups. The dataset is divided into a training set, for building your textual models, and a test set, which is comprised of posts that temporally follow the training set. You use the test set to test the accuracy of your predictions:

```
from sklearn.datasets import fetch_20newsgroups
```

```
newsgroups_train = fetch_20newsgroups(
```

```
    subset='train', remove=('headers', 'footers',
```

```
        'quotes'))
```

```
newsgroups_test = fetch_20newsgroups(
```

```
    subset='test', remove=('headers', 'footers',
```

```
        'quotes'))
```

After loading the two sets into memory, you import the two Naïve Bayes models and instantiate them. At this point, you set alpha values, which are useful for avoiding a zero probability for rare features (a zero probability would exclude these features from the analysis). You typically use a small value for alpha, as shown in the following code:

```
from sklearn.naive_bayes import BernoulliNB, MultinomialNB
```

```
Bernoulli = BernoulliNB(alpha=0.01)
```

```
Multinomial = MultinomialNB(alpha=0.01)
```



REMEMBER In some cases, you use what is known as the hashing trick to model textual data without fear of encountering new words when using the model after the training phase. (You can find the details of this technique in the “Performing the Hashing Trick” in Chapter 12 of *Python For Data Science For Dummies*, 2nd Edition, by John Paul Mueller and Luca Massaron [Wiley].) You can use two different hashing tricks, one counting the words (for the multinomial approach) and one recording whether a word appeared in a binary variable (the binomial approach). You can also remove *stop words*, that is, common words found in the English language, such as “a,” “the,” “in,” and so on.

```
import sklearn.feature_extraction.text as txt

multinomial = txt.HashingVectorizer(stop_words='english',

                                   binary=False, norm=None)

binary = txt.HashingVectorizer(stop_words='english',

                               binary=True, norm=None)
```

At this point, you can train the two classifiers and test them on the test set, which is a set of posts that temporally appear after the training set. The test measure is accuracy, which is the percentage of right guesses that the algorithm makes:

```
import numpy as np

target = newsgroups_train.target

target_test = newsgroups_test.target

multi_X = np.abs(

    multinomial.transform(newsgroups_train.data))

multi_Xt = np.abs(

    multinomial.transform(newsgroups_test.data))

bin_X = binary.transform(newsgroups_train.data)

bin_Xt = binary.transform(newsgroups_test.data)

Multinomial.fit(multi_X, target)

Bernoulli.fit(bin_X, target)
```

```

from sklearn.metrics import accuracy_score

for name, model, data in [('BernoulliNB', Bernoulli,
                           bin_Xt),
                          ('MultinomialNB', Multinomial,
                           multi_Xt)]:
    accuracy = accuracy_score(y_true=target_test,
                              y_pred=model.predict(data))

    print ('Accuracy for %s: %.3f' % (name, accuracy))

```

The reported accuracies for the two Naïve Bayes models are

```
Accuracy for BernoulliNB: 0.570
```

```
Accuracy for MultinomialNB: 0.651
```

You might notice that both models don't take long to train and report their predictions on the test set. Consider that the training set is made up of more than 11,000 posts containing more than 300,000 words, and the test set contains about 7,500 other posts.

```

print('number of posts in training: %i'
      % len(newsgroups_train.data))

D={word:True for post in newsgroups_train.data
   for word in post.split(' ')}

print('number of distinct words in training: %i'
      % len(D))

print('number of posts in test: %i'
      % len(newsgroups_test.data))

```

Running the code returns all these useful text statistics:

```
number of posts in training: 11314
```

```
number of distinct words in training: 300972
```

```
number of posts in test: 7532
```

Getting an overview of Bayesian inference

In reading about the thought experiment that Bayes originally used to create his theorem in the “[Delving into Bayes history](#)” section of the chapter, you discover the dynamic nature of the underlying technique. Each previous estimate acts as the basis for the next estimate — it’s a refinement. Essentially, dropping the second ball enables you to create a better estimate of the actual location of the first ball through the process of inference. Each piece of new data further improves the overall estimate so that the estimate begins to match real-world results. The following sections offer an overview of how inference can work to refine a Bayes probability.

Coming to the wrong conclusion

Often, people make assumptions about data that aren’t true. By making a single observation of the relationship between data elements under dubious conditions that could change, many people conclude that something is true when it isn’t. A single observation of the moon when it’s full may lead one to believe that the moon is always full, yet it isn’t. You need more observations to determine that the moon sometimes disappears altogether. To determine the current state of the moon, you then need enough observations to determine the moon’s cycle and then consider the progress of that cycle to properly conclude that the moon is currently at the first quarter — neither full nor new. Bayesian inference assumes the following:

- Data is limited to the observations made, which may not represent actual conditions.
- When modeling the data, you can easily fit the curve to the known observations a little too closely (overfitting) so that any conclusions about future data using the model are flawed.
- Some facts are more important than other facts, but the model often doesn’t contain this information, so making accurate predictions is impossible without knowing more about the facts.
- Conclusions are accurate only when you consider all the facts, rather than only the most likely fact.

Getting the required inference packages

Performing Bayesian inference requires the use of the PyMC3 (<https://pypi.org/project/pymc3/>) package, which relies on Markov chain Monte Carlo (MCMC) sampling to allow inference on increasingly complex models. You use this package for Bayesian statistical modeling in this book. To install the package, you begin by opening an Anaconda Prompt (don’t use a standard command prompt or terminal window because they won’t have the required environmental setup). At the prompt, you type the following (all on one line):

```
conda create -n PYMC3_env python=3.7.3 anaconda=2019.03
```

```
pymc3 nb_conda
```

and press Enter. The command will run for a while, seemingly doing nothing. However, at some point you see a description of what conda will do. Simply type **y** and press Enter to install the package.

To perform plotting using PyMC3, you also need the ArviZ package (<https://arviz-devs.github.io/arviz/>). Type these entries at the prompt to install this package:

```
activate PYMC3_env
```

```
conda install arviz
```

```
deactivate
```

As before, you need to type **y** and press Enter when prompted to perform the installation.

Developing a simple inference example

The “[Coming to the wrong conclusion](#)” section, earlier in this chapter, tells you about some assumptions for Bayesian inference that could seem mind boggling, so it’s time for an example. Consider that your company tasks you with determining how many of each car color to produce to ensure that your company doesn’t have an overstock. So, you go out to the parking lot and count 3 red cars, 2 black cars, and 1 tan car. (The example uses smaller numbers to make things simpler.) A simple solution would be to make 50 percent of the cars red, 33 percent of the cars black, and 17 percent of the cars tan, but the simple solution is probably wrong for these reasons (among others):

- You may not see all the cars driven by every employee.
- The numbers in your parking lot may not reflect reality.

To work through this problem, you need to make some assumptions and use some additional data. For example, you know that your company is interested in making only red, black, and tan cars, so you don’t need to consider other car colors in your analysis. You also discover that marketing has survey numbers that you can use to form a prior belief about the actual numbers. In this case, the survey shows that people actually prefer red cars in this ratio: for every 4 red car voters, 3 people voted for tan cars, and 1 person voted for a black car.

The technique used to create the initial (the prior) belief relies on the Dirichlet-multinomial distribution. You can read about it at <https://ixkael.github.io/dirichlet-multinomial-conjugate-priors/>, but this example seeks to simplify these ideas for those who really want to get some work done and don’t have a degree in statistics. The important aspects to consider are as follows:

- **Number of trials (observations):** Represented by n , which contains 6 in this case (3 red, 2 black, and 1 tan cars).

- **Number of outcomes:** Represented by k , which totals 3 in this case: one each for red, black, and tan.
- **A vector of probabilities for each outcome:** Represented by p in this case, this vector represents the output of the analysis.

The Dirichlet-multinomial distribution also relies on a special hyperparameter, α , which is a parameter of the prior. It contains the starting point information that exists before you make any observations, which would be the survey results of 4 red, 1 black, and 3 tan cars in this case. The survey simply represents an expectation — a guess. Because this is a survey, it's an educated guess, but it's still a guess. The code will contain two distributions: multinomial (parameters—event probabilities) and Dirichlet (prior distribution), as described here:

```
multinomial: (n=20, pred=?, pblack=?, ptan=?)
```

```
Dirichlet: (k=3,  $\alpha_{red}$ ,  $\alpha_{black}$ ,  $\alpha_{tan}$ )
```

You must also account for the support data, represented by c , which is an array containing the three observed values of 7 red cars (4 survey and 3 observed); 3 black cars (1 survey and 2 observed); and 4 tan cars (3 survey and 1 observed). The final model looks like this:

```
( $p$  |  $c$ ,  $\alpha$ )
```

Use the following code to calculate the probabilities for each outcome:

```
import numpy as np
```

```
colors = ['Red', 'Black', 'Tan']
```

```
c = np.array([3, 2, 1])
```

```
alphas = np.array([4, 1, 3])
```

```
expected_p = (alphas + c) / (c.sum() + alphas.sum())
```

```
new_values = dict(zip(colors, expected_p))
```

```
for x in new_values:
```

```
    print("{}:\t{:2.2%}".format(x, new_values[x]))
```

Based on the combination of the survey and the parking lot observation, you can now see that the percentages of colors to paint the cars has changed, like this:

Red: 50.00%

Black: 21.43%

Tan: 28.57%

As you continue to gather data, you can further refine the estimates, just as described for the original experiment. You must first create a model using the following code (which may generate deprecation notices depending on your setup):

```
import pymc3 as pm

with pm.Model() as model:
    parameters = pm.Dirichlet('parameters', a=alphas,
                               shape=3)
    observed_data = pm.Multinomial(
        'observed_data', n=6, p=parameters, shape=3,
        observed=c)
```

Note that `model` contains the two previously discussed distributions: `Dirichlet` and `Multinomial`. You can now train the model using this code:

```
with model:
    trace = pm.sample(draws=1000, chains=2, tune=500,
                      discard_tuned_samples=True)
```

The process samples data based on the No-U-Turn Sampler (NUTS), which is useful on models that have many continuous parameters. Basically, it searches for areas in the model where higher probabilities occur. The output you see explains how the sampling process occurs:

```
Auto-assigning NUTS sampler...
Initializing NUTS using jitter+adapt_diag...
Multiprocess sampling (2 chains in 4 jobs)
NUTS: [parameters]
Sampling 2 chains: 100%|██████████| 3000/3000
[00:11<00:00, 253.26draws/s]
```

NUTS uses `parameters`, which you previously defined as part of the model — the previously collected survey information. The progress bar shows the sampling process. If you're interested in the whole jitter+adapt_diag methodology, you can read about it at <https://discourse.pymc.io/t/what-exactly-is-jitter-adapt-diag-and-why-is-it-the-default-now/451>. Essentially, it's a sliding-window approach to interacting with the data.

At this point, you have a model that can predict with some level of precision how many cars to create in each of the colors. Of course, seeing the predictions is always nice, and you can do so with the following code:

```
dataPlot1 = pm.plots.traceplot(trace, combined=True)
```

```
dataPlot1[0][0].set_title(
```

```
    "Red Posterior Probability Distribution")
```

```
dataPlot1[0][1].set_title("Red Trace Samples")
```

```
dataPlot1[1][0].set_title(
```

```
    "Black Posterior Probability Distribution")
```

```
dataPlot1[1][1].set_title("Black Trace Samples")
```

```
dataPlot1[2][0].set_title(
```

```
    "Tan Posterior Probability Distribution")
```

```
dataPlot1[2][1].set_title("Tan Trace Samples")
```

The output from this code appears in [Figure 3-1](#). The graphs on the left show the probability distribution for each color, and the graphs on the right show the sample process used for each of the colors.

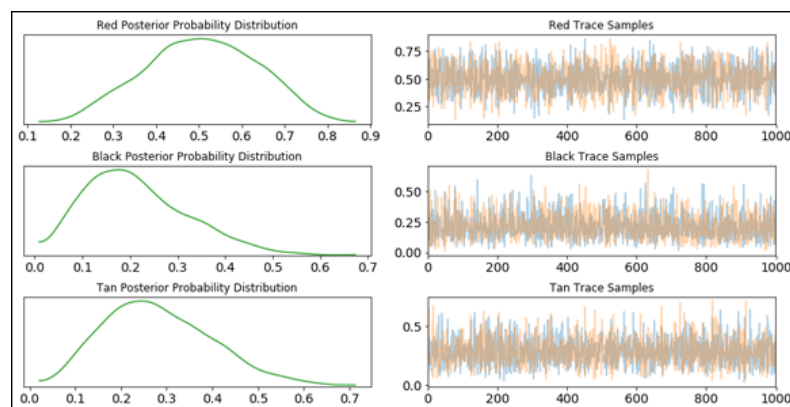


FIGURE 3-1: Seeing the probabilities for each of the colors.

All that remains is to determine the values to use and consider the probability that these values are correct by estimating the Highest Posterior Density (HPD) (you can read more about this estimation at

https://support.sas.com/documentation/cdl/en/statug/63033/HTML/default/viewer.htm#statug_introbay

The following code shows how to create the required plots:

```
dataPlot2 = pm.plots.plot_posterior(trace, figsize=(20,6))
```

```
dataPlot2[0].set_title('Red', fontsize=20)
```

```
dataPlot2[1].set_title('Black', fontsize=20)
```

```
dataPlot2[2].set_title('Tan', fontsize=20)
```

Figure 3-2 shows how the plots appear. Note that Red has a mean of 0.5 (50 percent), Black a mean of 0.2 (20 percent), and Tan a mean of 0.3 (30 percent). Each of these percentages reflects the number of cars to paint a certain color. The 94% HPD says that 94 percent of the estimates lie in the region shown, which is a high level of confidence that the calculation is correct.

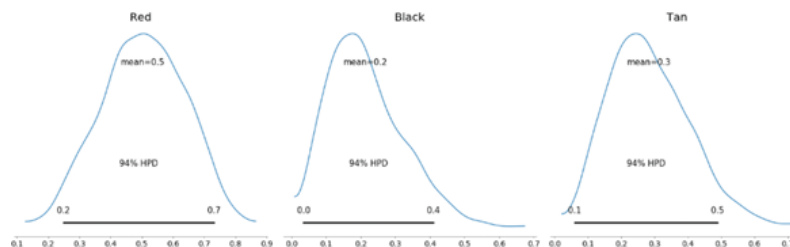


FIGURE 3-2: Determining how many cars to paint specific colors.

Moving forward with Bayes inference and deep learning

When performing data science tasks, you are often tasked with making algorithms work within specific fields such as AI learning, which relies on algorithms to define how and what to learn. In this environment, an *algorithm* is a kind of container. It provides a box for storing a method to solve a particular kind of a problem. Algorithms process data through a series of well-defined states. The states need not be deterministic, but the states are defined nonetheless. The goal is to create an output that solves a problem. In some cases, the algorithm receives inputs that help define the output, but the focus is always on the output.

Algorithms must express the transitions between states using a well-defined and formal language that the computer can understand. In processing the data and solving the problem, the algorithm defines, refines, and executes a function. The function is always specific to the kind of problem being addressed by the algorithm.

A group of scientists, called Bayesians, perceived that uncertainty was the key aspect to keep an eye on and that learning wasn't assured, but rather took place as a continuous updating of previous beliefs that grew more and more accurate. This perception led the Bayesians to adopt statistical methods and, in particular, derivations from Bayes' Theorem, which helps you calculate probabilities under specific conditions (for instance,

seeing a card of a certain *seed*, the starting value for a pseudo-random sequence, drawn from a deck after three other cards of same seed).

An application that relies on deep learning techniques uses Bayesian inference in a method called forward propagation (also called a feed-forward neural network) to help the neural network learn from the data you provide. In the article “Probabilistic Deep Learning: Bayes by Backprop,” at <https://medium.com/neuralspace/probabilistic-deep-learning-bayes-by-backprop-c4a3de0d9743>, you discover that the techniques shown in the previous sections also apply to deep learning, and many data scientists feel that such inference is essential to the future of deep learning.

However, techniques work with only specific applications, and Convolutional Neural Networks (CNNs) represent a specific application. The article “Bayesian Convolutional Neural Networks with Bayes by Backprop,” at <https://medium.com/neuralspace/bayesian-convolutional-neural-networks-with-bayes-by-backprop-c84dcaaf086e>, takes these ideas a step further. You can also discover more about both feed-forward techniques in Book 4, [Chapter 2](#), and about CNNs in Book 4, [Chapter 3](#). Using Bayes in the simple manner shown so far is just the starting point for significantly more complex techniques.

Working with Networked Bayes

Not all decisions come in neat packages that rely on just one prediction. Sometimes, you must consider multiple levels of decision making to better model a situation. Networked Bayes is a type of Probabilistic Graphical Model (PGM), which also includes the Markov Network. The difference between these two PGMs is that Bayes relies on a directed acyclic graph, while the Markov version is both undirected and cyclic. Both of these models model certain kinds of dependencies, but you use them in different ways and for different purposes. The following sections help you better understand networked Bayes and explain how you can use it to make predictions.

Considering the network types and uses

You use networked Bayes for a number of purposes that include prediction, anomaly detection, diagnostics, automated insight, reasoning, time series prediction, and decision making under uncertainty. However, you don’t use just one form of networked Bayes to perform all these tasks. Four different types or disciplines of networked Bayes exist, as described in the following list:

- **Descriptive:** Considers how to describe or categorize data based on the probability that it is one thing or another. This discipline has these characteristics and uses:
 - Automated insight
 - Large patterns
 - Anomalous patterns

- Multivariate
- **Diagnostic:** Specifies the cause of an error or determines whether something is useful based on the probability that the underlying assumptions about it are correct. This discipline has these characteristics and uses:
 - Value of information
 - Reasoning
 - Troubleshooting
 - Tracing anomalies
- **Predictive:** Defines a future value based on current information (as was done in the original experiment). This discipline has these characteristics and uses:
 - Supervised or unsupervised learning
 - Anomaly detection
 - Time series
 - Latent variables
- **Prescriptive:** Performs decision-making based on the truth-values of the underlying data and outputs these decisions with levels of confidence (based on probabilities). This discipline has these characteristics and uses:
 - Decision automation
 - Cost based decision-making
 - Decision support
 - Decision-making under uncertainty

One of the best ways to explore how you might use a networked Bayes setup is to play with it. You find an interactive version of the Asia Bayesian network (see [Figure 3-3](#)) at <https://www.bayesserver.com/examples/networks/asia>. By selecting the various boxes, you can see how each node of the network affects the overall probability of a particular disease to be detected in a person who visits a clinic. When you select a box, that particular condition becomes known and the probability is now 100 percent. If you leave the box deselected, the network uses an assumed set of percentages for true and false. Notice that the network has a definite hierarchy and a definite direction between decision-making steps.



FIGURE 3-3: The interactive version of the Asia Bayesian network is helpful in understanding how networks work.

When using graphs to represent a prediction model, you have a number of options from which to choose. As specified in the next section, networked Bayes relies on the directed acyclic graph. However, you need to know that graphs have a number of different characteristics, as specified in the following list:

- **Cyclic:** The edges forms a cycle that take you back to the initial vertex after having visited the intermediary vertexes.
- **A-cyclic:** This graph lacks cycles.
- **Directed:** Edges connect vertexes in a single direction, with an arrow or other indicator specifying the direction.
- **Undirected:** Edges connect vertexes in both directions (typically without arrows or other indicators).
- **Weighted:** Each edge has a cost associated with it, such as time, money, or energy, which you must pay to pass through it.
- **Unweighted:** All the edges have no cost or the same cost.
- **Dense:** This type of graph has a large number of edges when compared to the number of vertexes.
- **Sparse:** This type of graph has a small number of edges when compared to the number of vertexes.

Understanding Directed Acyclic Graphs (DAGs)

A *Directed Acyclic Graph (DAG)* is a finite directed graph that doesn't have any loops in it. In other words, you start from a particular location and follow a specific route to an ending location without ever going back to the starting location. When using topological sorting, a DAG always directs earlier vertexes to later vertexes. This kind of graph has all sorts of practical uses, such as schedules, with each milestone representing a particular milestone.

DAGs are one of the most important kinds of graphs because they see so many practical uses. The basic principles of DAGs are that they

- Follow a particular order so that you can't get from one vertex to another and back to the beginning vertex using any route.
- Provide a specific path from one vertex to another so that you can create a predictable set of routes.

You see DAGs used for many organizational needs. For example, a family tree is an example of a DAG. Even when the activity doesn't follow a chronological or other overriding order, the DAG enables you to create predictable routes, which makes DAGs easier to process than many other kinds of graphs you work with.

However, DAGs can use optional routes. Imagine that you're building a burger. The menu system starts with a bun bottom. You can optionally add condiments to the bun bottom, or you can move directly to the burger on the bun. The route always ends up with a burger, but you have multiple paths for getting to the burger. After you have the burger in place, you can choose to add cheese or bacon before adding the bun top. The point is that you take a specific path, but each path can connect to the next level in several different ways.



REMEMBER The DAG is commonly used with the evidence portion of a Bayes calculation. You use an evidentiary tree to decide various probabilities of evidence. For example, when deciding whether to play tennis, you might consider this tree of evidence:

1. Rain
2. Wind
3. Temperature
4. Humidity

If it isn't raining, the wind isn't too high, and the temperature is just right, it's a good day to play tennis. However, you must also consider all the other probabilities, which means calculating them for each branch of the tree. You don't need to consider any item more than once. When the prediction considers rain, it need not consider rain again. The tree also has an order, so you consider the chance of rain first. You see this particular tree in action in the "[Employing networked Bayes in predictions](#)" section, later in this chapter.



TIP When working with DAGs, you often see the nomenclature, $p(A, B)$, which means the probability of A and B . In other words, you no longer seek to find just the probability of A , but you want instead the probability of A and B occurring together, which means that they must both be true. You can see a conditional form of this nomenclature as well, $p(A, B | C)$, which means the probability of A and B both occurring given the evidence presented by C . To put this in a real-world context, you might want to consider the probability that it's both raining and windy given that you can see that the trees are bending an abnormal amount.

Sometimes you see this process extended further still by *instantiation*. In this case, you already know whether one of the items in the prediction is true or false. For example, you might see $p(A = \text{True}, B | C)$, which means the probability of B given that A is **True** and the evidence presented by C .

Employing networked Bayes in predictions

Bayes' Theorem can help you deduce how likely something is to happen in a certain context, based on the general probabilities of the fact itself and the evidence you examine, and combined with the probability of the evidence given the fact. Seldom will a single piece of evidence diminish doubts and provide enough certainty in a prediction to ensure that it will happen. As a true detective, to reach certainty, you have to collect more evidence and make the individual pieces work together in your investigation. Noticing that a person has long hair isn't enough to determine

whether person is female or a male. Adding data about height and weight could help increase confidence.

The Naïve Bayes algorithm helps you arrange all the evidence you gather and reach a more solid prediction with a higher likelihood of being correct. Gathered evidence considered singularly couldn't save you from the risk of predicting incorrectly, but all evidence summed together can reach a more definitive resolution. The following example shows how things work in a Naïve Bayes classification. This is an old, renowned problem, but it represents the kind of capability that you can expect from an AI. The dataset is from the paper "Induction of Decision Trees," by John Ross Quinlan (<https://dl.acm.org/citation.cfm?id=637969>). Quinlan is a computer scientist who contributed to the development of a machine learning algorithm called decision trees in a fundamental way, but his example works well with any kind of learning algorithm. The problem requires that the AI guess the best conditions to play tennis given the weather conditions. The set of features described by Quinlan is as follows:

- **Outlook:** Sunny, overcast, or rainy
- **Temperature:** Cool, mild, hot
- **Humidity:** High or normal
- **Windy:** True or false

The following table contains the database entries used for the example:

<i>Outlook</i>	<i>Temperature</i>	<i>Humidity</i>	<i>Windy</i>	<i>PlayTennis</i>
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Overcast	Hot	High	False	Yes
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Rainy	Mild	Normal	False	Yes
Sunny	Mild	Normal	True	Yes

<i>Outlook</i>	<i>Temperature</i>	<i>Humidity</i>	<i>Windy</i>	<i>PlayTennis</i>
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Rainy	Mild	High	True	No

The option of playing tennis depends on the four arguments shown in [Figure 3-4](#).

The result of this AI learning example is a decision as to whether to play tennis, given the weather conditions (the evidence). Using just the outlook (sunny, overcast, or rainy) won't be enough, because the temperature and humidity could be too high or the wind might be strong. These arguments represent real conditions that have multiple causes, or causes that are interconnected. The Naïve Bayes algorithm is skilled at guessing correctly when multiple causes exist.

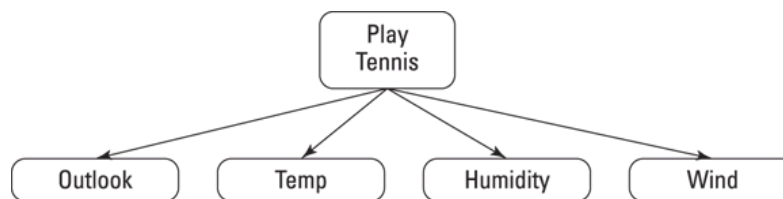


FIGURE 3-4: A Naïve Bayes model can retrace evidence to the right outcome.

The algorithm computes a score, based on the probability of making a particular decision and multiplied by the probabilities of the evidence connected to that decision. For instance, to determine whether to play tennis when the outlook is sunny but the wind is strong, the algorithm computes the score for a positive answer by multiplying the general probability of playing (9 played games out of 14 occurrences) by the probability of the day's being sunny (2 out of 9 played games) and of having windy conditions when playing tennis (3 out of 9 played games). The same rules apply for the negative case (which has different probabilities for not playing given certain conditions):

likelihood of playing: $9/14 * 2/9 * 3/9 = 0.05$

likelihood of not playing: $5/14 * 3/5 * 3/5 = 0.13$

Because the score for the likelihood is higher, the algorithm decides that it's safer not to play under such conditions. It computes such likelihood by summing the two scores and dividing both scores by their sum:

probability of playing : $0.05 / (0.05 + 0.13) = 0.278$

probability of not playing : $0.13 / (0.05 + 0.13) = 0.722$

Look at this example in a little more detail. The dataset is quite simple, consisting of only 14 observations relative to the weather conditions, with results that say whether playing tennis is appropriate.

The example contains four features: outlook, temperature, humidity, and wind, all expressed using qualitative classes instead of measurements (you could express temperature, humidity, and wind strength numerically) to convey a more intuitive understanding of how the weather features relate to the outcome. After these features are processed by the algorithm, you can represent the dataset using a tree-like schema, as shown in [Figure 3-5](#). As the figure shows, you can inspect and read a set of rules by splitting the dataset to create parts in which the predictions are easier by looking at the most frequent class (in this case, the outcome, which is whether to play tennis).

To read the nodes of the tree, just start from the topmost node, which corresponds to the original training data; next, start reading the rules. Note that each node has two derivations: The left branch means that the upper rule is true (stated as yes in a square box), and the right one means that it is false (stated as no in a square box).

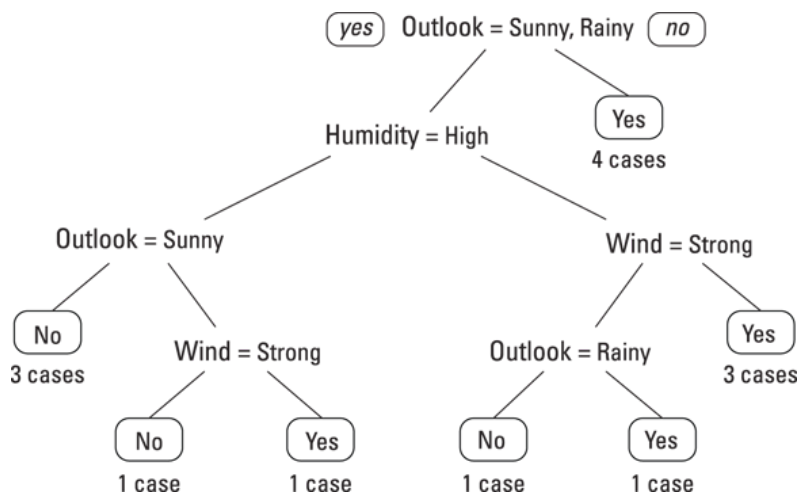


FIGURE 3-5: A visualization of the decision tree built from the play-tennis data.

On the right of the first rule, you see an important terminal rule (a terminal leaf), in a circle, stating a positive result, Yes, that you can read as play tennis=True. According to this node, when the outlook isn't sunny (Sun) or rainy (Rain), it's possible to play. (The numbers under the terminal leaf show four examples affirming this rule and zero denying it.) Note that you could understand the rule better if the output simply stated that when the outlook is overcast, play is possible. Frequently, decision tree rules aren't immediately usable, and you need to interpret them before use. However, they are clearly intelligible (and much better than a coefficient vector of values).

On the left, the tree proceeds with other rules related to Humidity. Again, on the left, when humidity is high and outlook is sunny, most terminal leaves are negative, except when the wind isn't strong. When you explore

the branches on the right, you see that the tree reveals that play is always possible when the wind isn't strong, or when the wind is strong but it doesn't rain.

Bayesian networks, though intuitive, have complex math behind them, and they're more powerful than a simple Naïve Bayes algorithm because they mimic the world as a sequence of causes and effects based on probability. Bayesian networks are so effective that you can use them to represent any situation. They have varied applications, such as medical diagnoses, the fusing of uncertain data arriving from multiple sensors, economic modeling, and the monitoring of complex systems such as a car. For instance, because driving in highway traffic may involve complex situations with many vehicles, the Analysis of Massive Data Streams (AMIDST) consortium, in collaboration with the automaker Daimler, devised a Bayesian network that can recognize maneuvers by other vehicles and increase driving safety. You can read more about this project and see the complex Bayesian network at

<http://project.amidsttoolbox.com/use-cases/identification-and-interpretation-of-maneuvers-in-traffic.html>.

Deciding between automated and guided learning

The tree structure used throughout the chapter must have some sort of start. You must either guide the creation of the tree structure, or you must allow for automated creation. A guided method often relies on

- Historic data
- Prior expert knowledge (a guess based on experience)
- Premodeled data created through various means, such as experimentation or previous analysis

An automated method takes a different route. Rather than rely on human input, this route reviews the data and automatically generates the tree. You can divide this approach into two tasks:

- **Structure learning:** Create a tree structure based on the connectivity expressed in the data.
- **Parameter learning:** Given a particular kind of tree structure, calculate the parameters used to create it.

These kinds of learning see use when working with extremely complex data. You can read about two significant uses of these learning approaches:

- Comparison of automatic and guided learning for Bayesian networks to analyze pipe failures in the water distribution system (<https://www.sciencedirect.com/science/article/pii/S0951832017309377>)
- Construction and application of Bayesian networks in flood decision supporting system (<https://ieeexplore.ieee.org/abstract/document/1174468>)

Considering the Use of Bayesian Linear Regression

Linear regression used on its own is a simple prediction methodology that relies on the use of data points, as described in [Chapter 1](#) of this minibook. The problem is that the technique doesn't work well when you have insufficient data or suffer from a poor data distribution. Bayesian linear regression can overcome these problems by using the priors on the coefficients and noise so that in the absence of data, the priors can take over. More important, you can now use Bayes to determine the confidence level of the data in the following ways:

- Calculate the confidence level of the estimated linear relation
- Determine the full posterior distribution
- Assess the estimated noise level
- Assess the estimated gradient
- Determine the direction of steepest ascent
- Discover the location of an optimum or saddle-point

**TIP**

This particular technique enables you to fill in missing data. You can see the effect of this strategy demonstrated in the articles found at <https://towardsdatascience.com/bayesian-linear-regression-in-python-using-machine-learning-to-predict-student-grades-part-1-7d0ad817fca5> and <https://towardsdatascience.com/bayesian-linear-regression-in-python-using-machine-learning-to-predict-student-grades-part-2-b72059a8ac7e>.

Considering the Use of Bayesian Logistic Regression

Logistic regression helps you make predictions based on one or more inputs, as described in [Chapter 2](#) of this minibook. For example, it could predict whether a person will get cancer based on weight and smoking habits. It works on discrete values, versus the continuous values used by linear regression. Adding Bayes to the mix helps ascertain the confidence level of a particular analysis. For example, it's now possible to say that if someone smokes and is overweight, they will get cancer with a confidence factor of so many percent. However, it goes further than that by assigning a prior to each of the independent values, giving them a weight of a sort in the calculation. With this in mind, here are the benefits of using Bayesian logistic regression:

- The output consists of a range of inferential solutions, rather than a point estimate.
- Starts with prior information, rather than just a belief in the calculation.
- Updating the prediction becomes possible based on changing evidence.
- Reduces potential bias in a prediction based on features that may not significantly contribute to an outcome.
- Potentially improves the overall accuracy of a prediction.



REMEMBER It's important to know that these advantages are crucial in certain fields, such as medicine. The article at <https://academic.oup.com/aje/article/153/12/1222/124010> discusses the use of Bayesian logistic regression in improving the accuracy of disease predictions for situations in which some features have a small probability (such as the effect of smoking on fetus health). You can see an example of building a Bayesian logistic regression using Python and PyMC3 at <https://towardsdatascience.com/building-a-bayesian-logistic-regression-with-python-and-pymc3-4dd463bbb16>.

Table of contents collapsed